

		ITER, SIKSHA 'O' ANUSANDHAN (Deemed to be University)		Assignment 3		
Branch		CSE	Programme		B.Tech	
Course Name		Computer Science Workshop 2	Semester		4th	
Course Code		CSE 3141	Academic Year		2025-2026	
Assignment: 3			Topic: Enhancing Your Blog and Adding Social Features			
Learning Level (LL)	L1: Remembering		L3: Applying		L5: Evaluating	
	L2: Understanding		L4: Analysing		L6: Creating	
Q's	Questions				COs	LL
Q1.	Enhance the existing Post model by implementing canonical URL functionality. Define a method named get_absolute_url() inside the Post model that dynamically constructs the URL for a post using Django's reverse() function. Modify the template files so that links to individual blog posts use this canonical URL method instead of manually constructing URLs. After implementing the method, verify that each post correctly redirects to its respective detail page. Expected Outcomes - Canonical URL method implemented in the model - Post detail pages open correctly				CO1	L2, L3
Q2.	Modify the existing URL routing mechanism of the blog application to create SEO-friendly URLs. Update the URL pattern of the post detail view so that the URL format becomes: /year/month/day/slug/ Update the post_detail view to retrieve the post using the publish date and slug. Also modify the slug field in the Post model using the parameter: unique_for_date='publish' Apply the required migrations and verify that posts can be accessed using the new URL structure. Expected Outcomes - URL contains publish date and slug - Post detail pages accessible through SEO-friendly URLs				CO1	L3, L4
Q3.	Enhance the blog application by implementing pagination for the post list page. Use Django's Paginator class to divide the list of published posts into pages with three posts per page. Modify the view to retrieve the requested page number using the GET parameter page. Additionally, implement exception handling to manage invalid page numbers				CO1	L3, L4

	such as non-integer values or pages that are out of range. Create a reusable template named <code>pagination.html</code> to display navigation links. Expected Outcomes • Blog posts divided into multiple pages • Navigation links (Previous / Next) displayed • Invalid page requests handled correctly		
Q4.	Replace the existing function-based post_list view with a class-based view using Django's <code>ListView</code>. Configure the class with the following attributes: • <code>queryset</code> to retrieve only published posts • <code>context_object_name</code> as <code>posts</code> • <code>paginate_by</code> set to 3 • <code>template_name</code> for rendering the blog list page Update the URL configuration to use the class-based view. Expected Outcomes • Blog list implemented using <code>ListView</code> (Class-based architecture) • Pagination continues to function correctly	CO2	L3, L4
Q5.	Develop a feature that allows users to share blog posts through email. Create a form named EmailPostForm with the following fields: • sender name • sender email • recipient email • optional comments Develop a view named post_share that processes the form submission and sends an email using Django's <code>send_mail()</code> function. Configure SMTP settings in <code>settings.py</code> and render the form in a template named share.html. Expected Outcomes • Form for sharing blog posts created • Email successfully sent to recipient • Success message displayed after submission	CO2	L3, L4
Q6.	Extend the blog application by creating a Comment model to allow users to comment on posts. The model should contain the following fields:	CO2	L3, L4

	• post (ForeignKey to Post model) • name • email • body • created • updated • active Apply database migrations and register the Comment model in the Django administration site. Expected Outcomes • Comment model created and migrated • Comments stored in the database • Admin panel displays comment records		
Q7.	Create a form named CommentForm using Django's ModelForm class. The form should dynamically generate fields from the Comment model and include only the following fields: name, email, and body Develop a view named post_comment to process comment submissions and store the comments in the database. Expected Outcomes • Comment form created using ModelForm • Comments submitted and saved in database	CO3	L2, L3
Q8.	Enhance the post detail page to display: • all comments related to the post • a form for adding new comments Use the {% include %} template tag to include the comment form template within the post detail page. Ensure that only active comments are displayed. Expected Outcomes • Comment list displayed under each post • Users can add comments from the detail page • Comment system integrated with blog application	CO3	L3, L4
Q9.	Develop a Django project named article_project and create an application named articleApp within the project. Configure the application by activating it in the INSTALLED_APPS section of settings.py. Further, design a basic Article model inside models.py containing the fields title (CharField with max_length=200), slug (SlugField), content (TextField), and publish (DateTimeField with	CO3	L3, L4

	default=timezone.now). Implement the <code>__str__()</code> method to return the title of the article. Create a view named article_list that retrieves all articles and displays them in a template named article_list.html. Implement pagination using Django's Paginator class so that only five articles appear per page. Configure URL routing so that visiting <code>/articles/</code> displays the paginated list of articles. Expected Outcomes: • Visiting <code>/articles/</code> displays the article list • Pagination navigation (Previous / Next) works correctly • Articles appear in multiple pages when the number exceeds five		
Q10.	Enhance the ArticleApp application by implementing user interaction features. Create a form named ArticleShareForm using Django's Form class containing the fields: name, email, recipient email, and comments. Develop a view named article_share that processes the form submission and sends article recommendations using Django's <code>send_mail()</code> function. Further, implement a Comment model associated with the Article model using a ForeignKey relationship. Create a CommentForm using ModelForm and allow users to submit comments for each article from the article detail page. Ensure that comments are saved in the database and displayed below the article content. Expected Outcomes: • Article sharing form working • Email recommendation sent successfully • Users can submit comments for articles • Comments displayed on the article detail page	CO3	L4, L6
	-END-		

Note:

1. Assignment carries a weightage of 10% **marks out of 100**
2. Only CO1, CO2, and CO3 outcomes are covered.

Course Outcomes	CO1	Remember and identify the fundamental concepts of web development and Django framework.
	CO2	Explain and interpret Django workflows such as authentication, template rendering, and form handling.
	CO3	Apply Django concepts to develop functional web applications.
	CO4	Implement advanced Django features including APIs, payments, internationalization, and real-time systems.
	CO5	Analyze Django applications with respect to caching strategies, API design, data flow, and performance-related design considerations.
	CO6	Analyze and design an end-to-end Django solution including deployment and system integration.